

SkeletonSR (BasicSR): Pseudocode

Overview. A minimal PySR-style symbolic-regression engine. Expressions are trees of unary/binary operators over variables and constants. The skeleton owns the tree representation, the evaluation budget and the outer loop. The *policy* (in **BLUE**) decides how expressions are scored, chosen, varied, accepted and kept; BasicSR supplies **LOSS_FUNCTION**, **SELECTION**, **MUTATION**, **CROSSOVER**, **ACCEPTANCE**, **SURVIVAL**, **UPDATE_POPULATION** and **UPDATE_STATE!**. Every call to **LOSS_FUNCTION** costs one evaluation; the search stops when HASBUDGET (evals < max_evals and elapsed < time-out) fails.

```

1: function FITSKELETONSR( $X, y$ , config)
2:   policy_state  $\leftarrow$  init_state(config)
3:   populations  $\leftarrow n_{\text{pops}}$  populations of population_size small random trees,
      each scored with LOSS_FUNCTION  $\triangleright$  Const(0) with 3 random operators attached
4:   UPDATE_STATE!(populations)
5:   for iteration = 1 ... niterations do
6:     for each population  $P$  do
7:       for 1 ... ncycles_per_iteration do
8:         UPDATE_STATE!(populations)
9:         for 1 ...  $\lceil |P|/\text{tournament}_n \rceil$  do
10:          if rand  $\leq$  crossover_probability then
11:             $a \leftarrow$  SELECTION( $P$ );  $b \leftarrow$  SELECTION( $P$ ) (distinct)
12:            children  $\leftarrow$  [score( $t$ ) for  $t$  in CROSSOVER( $a, b$ )], keep those passing ACCEP-
TANCE(parent, child)
13:            if children non-empty then  $P \leftarrow$  SURVIVAL( $P$ , children)
14:          else
15:            parent  $\leftarrow$  SELECTION( $P$ )
16:            child  $\leftarrow$  score(MUTATION(parent))  $\triangleright$  skip if mutation returns  $\emptyset$ 
17:            if ACCEPTANCE(parent, child) then  $P \leftarrow$  SURVIVAL( $P$ , [child])
18:          UPDATE_STATE!(populations)
19:          populations  $\leftarrow$  UPDATE_POPULATION(populations)  $\triangleright$  e.g. migration, hall of fame
20:   return policy_state.archive sorted by complexity

```

where $\text{score}(t) = \text{Individual}(t, \text{LOSS_FUNCTION}(t), \text{complexity} = \text{size}(t))$, and every loop above also exits early once HASBUDGET fails.